



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



UCSC



UNIVERSIDAD DEL BÍO-BÍO



Inference Optimizer

Knowledge distillation and NVIDIA TensorRT

Part II

Doctorado en Inteligencia Artificial

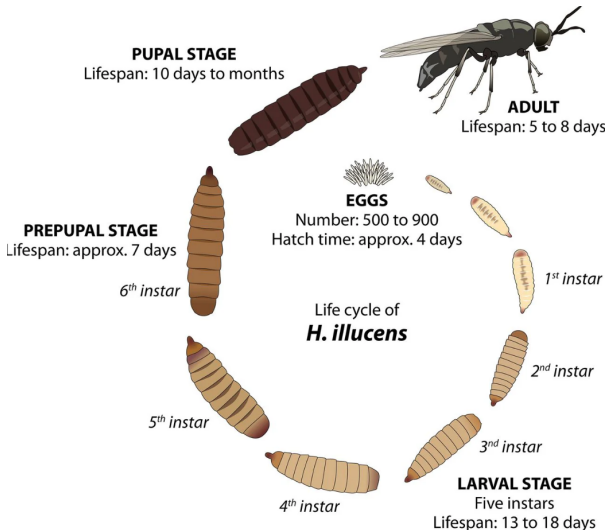
Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

1 Specialized Computing

- TinyML
- HLS4ML
- Xilinx Vitis AI

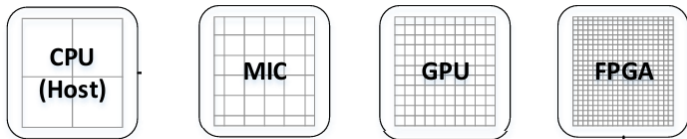
2 TensorRT



*"Many insects have a larval form that is optimized for extracting energy and nutrients from the environment and a completely different adult form that is optimized for the very different requirements of traveling and reproduction."*¹

¹G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network", arXiv preprint arXiv:1503.02531, 2015.

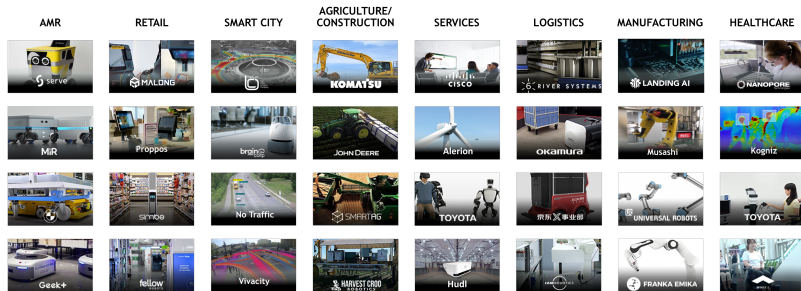
Specialized Computing



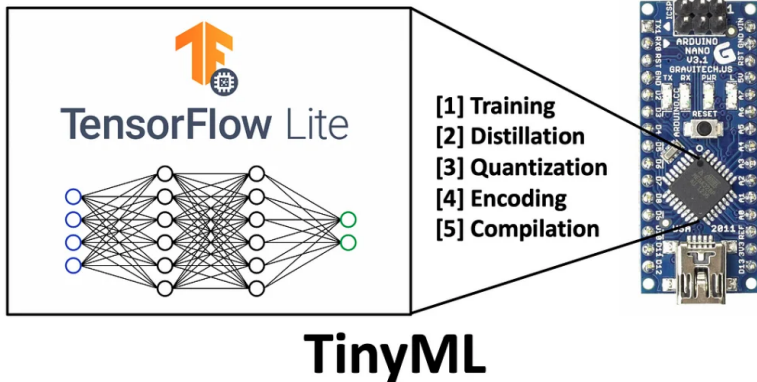
- ▶ La *Central Processing Unit* (CPU) convencional, junto con cualquier combinación de aceleradores especializados que ofrecen granularidad mixta para procesamiento paralelo².
- ▶ Los aceleradores especializados incluyen:
 - **Many Integrated Cores (MIC)** processors³.
 - **Graphics Processing Units (GPUs)**.
 - **Field-Programmable Gate Arrays (FPGAs)**.
- ▶ Estas tecnologías representan la mayor parte de la innovación en hardware disponible hoy en día para aplicaciones de ML.

²La unidad de cómputo es capaz de dividir las tareas en porciones más pequeñas y procesarlas en paralelo.

³Intel Xeon Phi, son procesadores diseñados para el cómputo paralelo masivo. Su arquitectura se basa en múltiples cores para procesar tareas de manera simultánea.

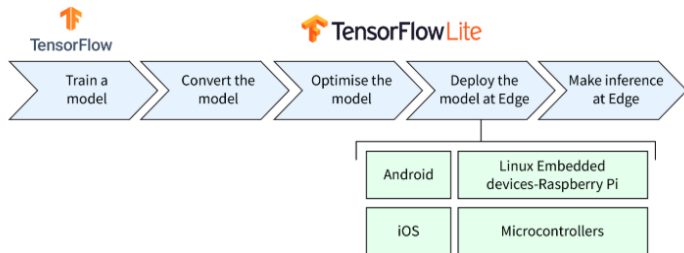


- ▶ *Safe-critical*
- ▶ *Resource-constrained devices*

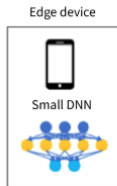


Limitaciones de memoria, latencia en tiempo real y consumo de energía

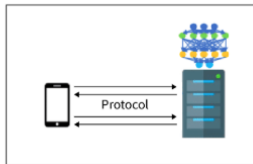
⁴Weight encoding: codificar los pesos de forma eficiente para reducir tamaño y mejorar velocidad de carga.



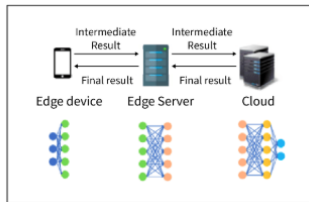
- ▶ El paradigma TinyML busca habilitar la implementación de algoritmos de deep learning para aplicaciones de análisis de datos “on-device”.
- ▶ Énfasis en optimizaciones para dispositivos de bajo consumo energético en aplicaciones que requieran autonomía, computo local para preservar la privacidad de los datos y que sean sensibles al costo.
- ▶ Los campos de aplicación principales para TinyML incluyen sensores inteligentes e infraestructura para el *Internet of Things*.



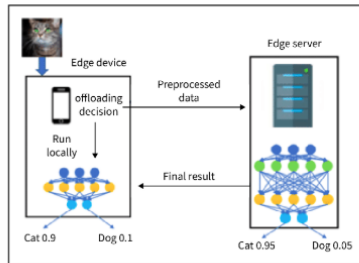
(a)



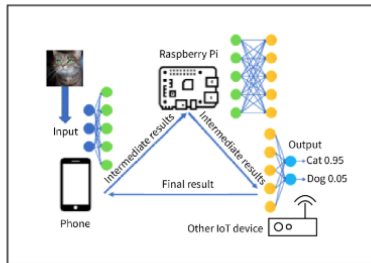
(b)



(c)



(d)

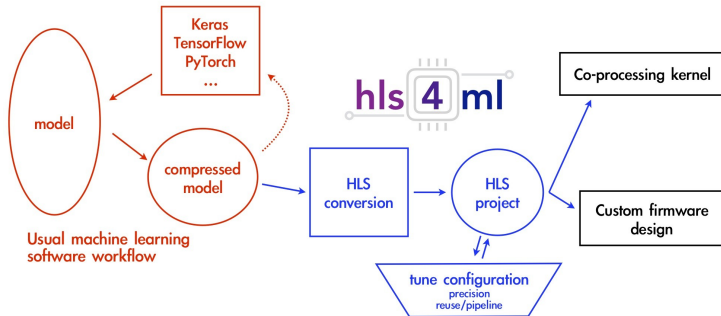


(e)



`hls4ml` is a Python package for machine learning inference in FPGAs. We create firmware implementations of machine learning algorithms using a *high-level synthesis* language (HLS).

`hls4ml` translate traditional open-source ML package models into HLS that can be configured for your use-case.



- ▶ Vitis AI development environment
 - Edge devices
 - Zynq-7000 SoCs, Zynq Ultrascale + MPSoCs, Versal AI Core Series.
 - Cloud
 - Alveo accelerator cards
- ▶ Optimized Core IPs known as Deep Learning Processing Units (DPU)⁵
- ▶ ModelZoo → more than 100 pre-trained models, ready to deploy on Xilinx devices.
 - Find the closest model and start retraining.
- ▶ Dynamic functionality on an FPGA:
 - Can be partially reconfigured.

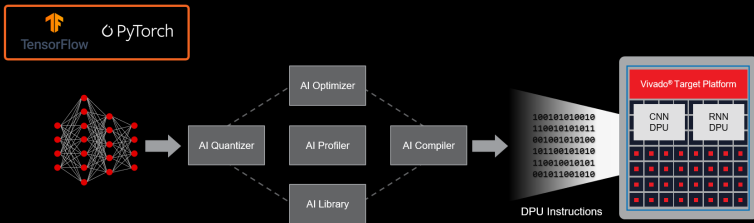
Tutoriales: <https://www.youtube.com/watch?v=Cievqcw056g>

⁵ Acelerador para *machine learning*.

Vitis AI development environment

- ▶ **AI Optimizer:** Can prune a model by up to 90 %.
- ▶ **AI Quantizer:** Support model quantization, calibration and fine tuning.
- ▶ **AI Profiler:** Provide layer-by-layer analysis to help with bottlenecks.
- ▶ **AI Library:** Pretrained AI models.
- ▶ **AI Compiler:** Maps the model to high-efficient instruction set (layer fusion and scheduling).

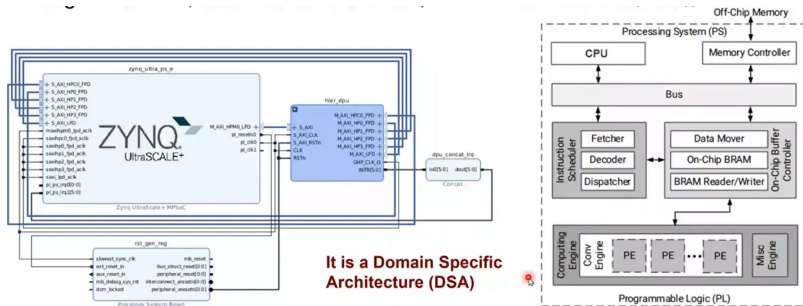
Deployment in the Vitis™ AI Environment – A 1000 Foot View



Build and Deploy AI to All Platforms

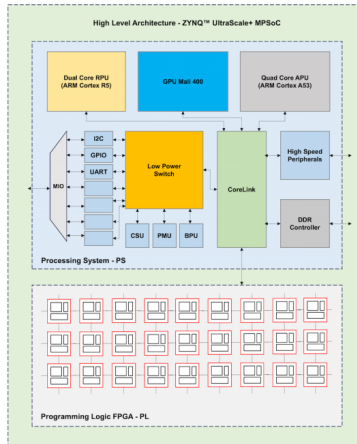
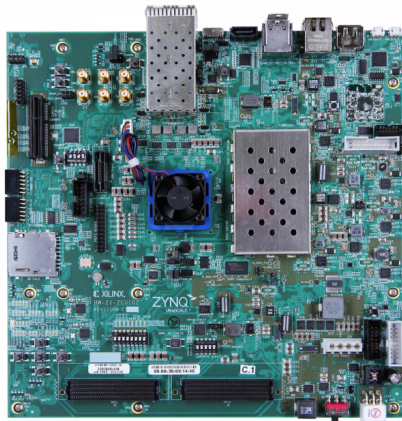


DPU is a programmable engine dedicated for CNN. The unit contains register configure module, data controller module, and convolution computing module.

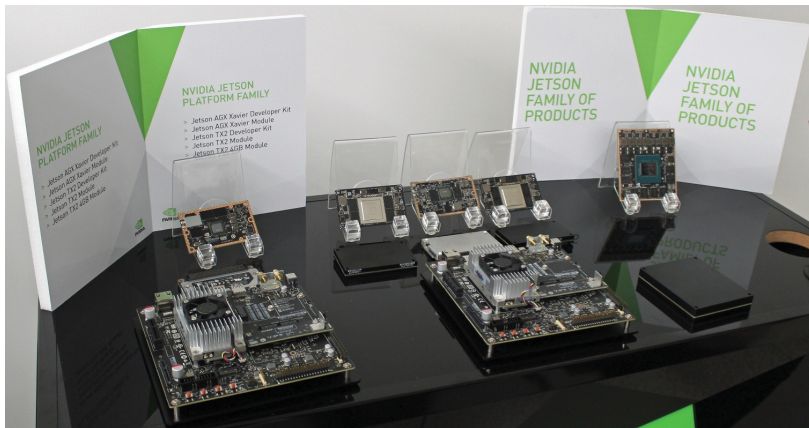


The Deployed CNNs in DPU includes VGG, ResNet, GoogleNet, YOLO, SSD, MobileNet, etc.

Zynq UltraScale+ MPSoC ZCU104 Evaluation Kit

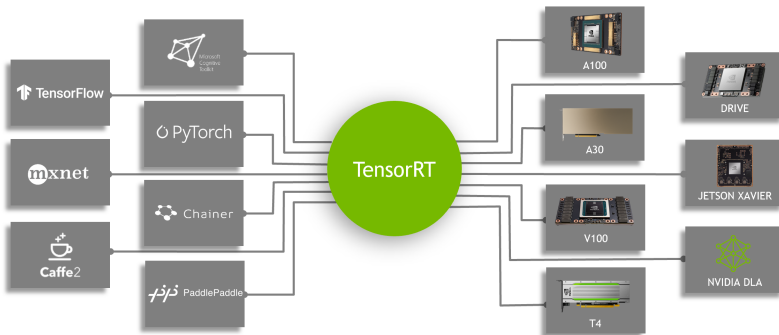


Jetson - Características de los dispositivos

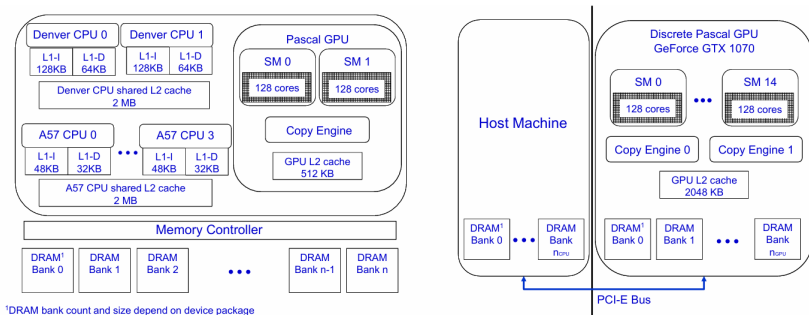


Plataforma	Núcleos GPU	Memoria (GB)	CPU
Nvidia GeForce RTX 3060	—	12 (GDDR6) + 36 RAM	Intel i3-12100F
Nvidia GeForce RTX 2060	—	6 (GDDR6) + 12 RAM	Intel i7-10750H
Jetson Xavier NX	384	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.9 GHz
Jetson Xavier AGX	512	16 RAM (compartida)	ARMv8.2, 8 núcleos, 2.3 GHz
Jetson Orin Nano	1024	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.7 GHz
Jetson Orin AGX	2048	64 RAM (compartida)	ARMv8.2, 12 núcleos, 2.2 GHz

TensorRT - Inference Optimizer



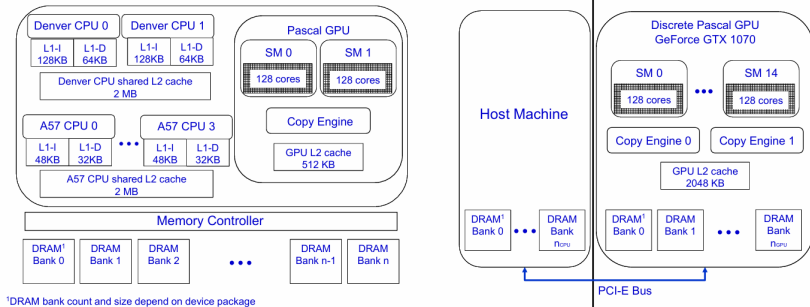
- ▶ CUDA es una plataforma computacional y modelo de programación desarrollado por Nvidia.
- ▶ Permite ejecutar tareas altamente paralelizables en GPUs de Nvidia.
- ▶ Se basa en el concepto de *General-Purpose Computing on GPUs* (GPGPU).
- ▶ Compatible con lenguajes como C/C++, Fortran y Python.



- Diagrama de comunicación entre *host* y *device*: GPU integrada (izquierda) y GPU discreta (derecha)⁷
- CUDA es un modelo de programación heterogéneo que utiliza tanto la CPU como la GPU.

⁷Ming Yang, et al, "Avoiding Pitfalls when Using NVIDIA GPUs for Real-Time Tasks in Autonomous Systems," *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2021.

Elementos del Procesamiento Paralelo en GPUs



► Implementación *system-on-chip* (SoC).

- Esta integra la GPU, los *CPU Cores*, la memoria (*Unified Memory*) y los conectores externos en una pequeña computadora de una sola placa. La GPU integrada comparte recursos de hardware, como la memoria, con los *CPU Cores*.

► GPU discreta

- GPUs discretas, consisten únicamente en los *Streaming Multiprocessors* (SMs) y la memoria local del dispositivo (*device memory*).
- Empaquetados en una tarjeta para su instalación en una ranura de expansión PCIe de la placa base de una computadora.

Niveles populares de software y hardware.

Capa	Ejemplos
Aplicación	Visión por computadora Control Automático Reconocimiento de voz Conducción autónoma
Modelo	ResNet - MobileNet - MobileViT YOLO - U-Net
Framework	PyTorch - TensorFlow - Keras
Graph Format	ONNX NNEF ⁸
Inference Optimizer	TensorRT ARM Compute Library Intel OpenDNN AMD ML Open Xilinx Vitis AI
Hardware	GPU CPU FPGA Aceleradores especializados (TPU, NPU, etc.)

⁸Sintaxis similar a JSON o Python, lo que facilita su comprensión y edición manual

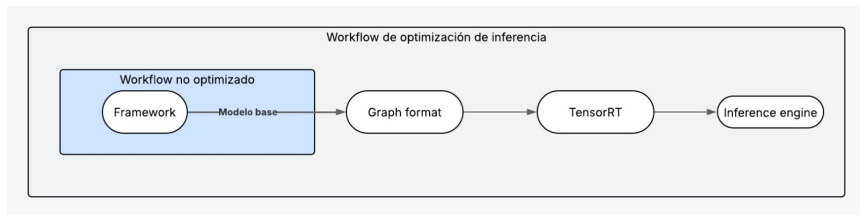
TensorRT

Un inference optimizer es una herramienta o conjunto de técnicas utilizadas para mejorar la eficiencia y el rendimiento de un modelo de *machine learning* durante la fase de inferencia.

- ▶ Una vez que el modelo ha sido entrenado, puede ser procesado por un **inference optimizer**:
 - Compresión del modelo y mapeo de operaciones al hardware disponible.
 - Se genera el **inference engine** → información de como ejecutar operaciones del modelo ML en el hardware objetivo.

Workflow no optimizado: no considera las particularidades de la arquitectura de *Deep Learning*, siendo agnóstico a ella.

No optimizado para una arquitectura específica, lo que implica desconocimiento del hardware subyacente y de sus capacidades.



⁹ Juan Carlos Aguilera. UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA DEPARTAMENTO DE ELECTRÓNICA EXPLORACIÓN DE METODOLOGÍAS PARA LA OPTIMIZACIÓN DE INFERENCIA DE REDES NEURONALES EN GPU UTILIZANDO TENSORRT. Master's thesis, Universidad Técnica Federico Santa María, 2024

Algunos ejemplos de flujos de trabajo de optimización de inferencia reportados en la literatura reciente:

- ▶ **PyTorch - TensorRT:** Describe el proceso proporcionado por PyTorch para optimizar modelos utilizando TensorRT.
- ▶ **TensorFlow - TensorRT:** Aquí, el formato de grafo también está integrado dentro de TensorFlow.
- ▶ **PyTorch - ONNX - TensorRT:** Detalla la transición de un modelo al formato ONNX, que luego se optimiza utilizando TensorRT.

- ▶ TensorRT es un *Software Development Kit* (SDK) para generar *inference engines* optimizados.
- ▶ Diseñado para inference a partir de modelos preentrenados.
- ▶ Optimizaciones basadas en propiedades específicas del hardware de Nvidia.
- ▶ Es software cerrado, y los detalles internos no son públicos.
- ▶ La documentación oficial de Nvidia y estudios académicos ofrecen descripciones generales de sus optimizaciones.
- ▶ Se integra con frameworks populares de machine learning.
- ▶ Existe evidencia empírica sustancial sobre los *inference engines* generados por TensorRT.

- ▶ TensorRT proporciona APIs para C++ y Python.
- ▶ Permiten programar una **TensorRT Application**, que es una implementación de un inference optimizer mediante las funciones proporcionadas por TensorRT.
- ▶ Una TensorRT Application puede optimizar modelos y realizar inferencias.
- ▶ Los pasos para implementar una TensorRT Application se describen en la documentación oficial de TensorRT, pero no de manera exhaustiva.
- ▶ Los pasos específicos pueden variar según la aplicación, el nivel de optimización, el framework utilizado, etc.

- ▶ El flujo de TensorRT Application comienza con un **modelo base**.
- ▶ Sigue una **building phase** donde se optimiza la red para una GPU específica.
- ▶ Luego, en la **running phase**, se utiliza el modelo optimizado para ejecutar inferencias.
- ▶ Hay documentación y ejemplos en la web a fin de facilitar la adopción y el uso de la herramienta, abarcando desde decisiones de diseño hasta la reproducibilidad de resultados.

Flujo de diseño de TensorRT

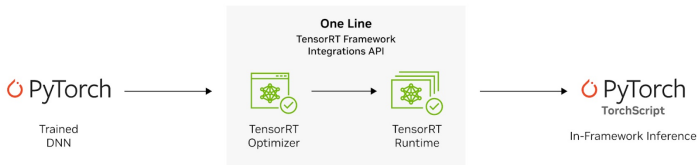
Pipeline de tres etapas principales:

1. Building phase.
2. Serialización del Engine¹⁰
3. Fase de Ejecución (Running Phase)

TensorRT



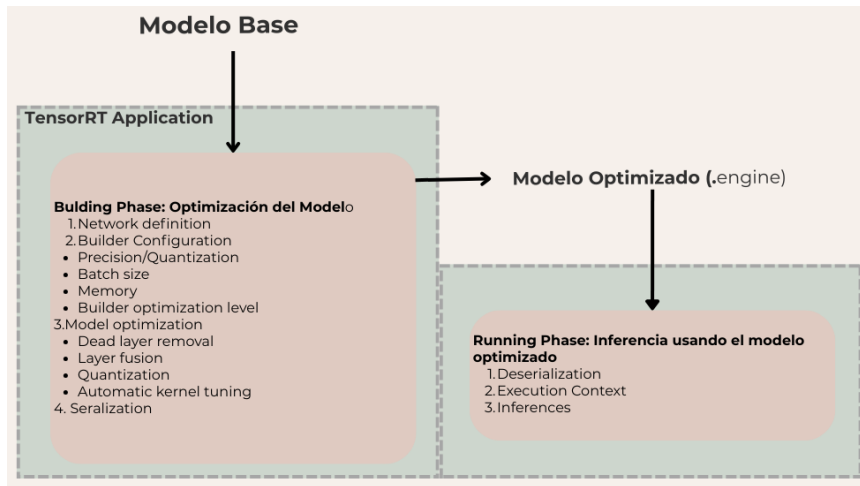
TensorRT Framework Integrations



¹⁰Guardar la representación optimizada (engine) en un archivo binario, que luego puede cargarse para hacer inferencias sin necesidad de recompilar u optimizar de nuevo.

Building phase

En el **building phase**, TensorRT desempeña el papel de builder, encargado de optimizar la red y producir un *inference engine*.



- ▶ Previo a la optimización, es necesario crear una configuración del builder.
- ▶ Esta configuración se puede modificar para personalizar la optimización del modelo.
- ▶ El ajuste de la configuración puede afectar el uso de recursos, la calidad de los resultados y la viabilidad del proceso de optimización.
- ▶ Los parámetros configurables están enumerados en la documentación *online*.

Precision/quantization

- ▶ Controla cómo se representan los valores numéricos de los parámetros del modelo.
- ▶ Mayor precisión (fp32):
 - Garantiza cálculos más exactos.
 - Aumenta el consumo de recursos computacionales.
- ▶ TensorRT permite reducir la precisión a fp16 o cuantizarlos a int8.
- ▶ La cuantización puede mejorar la eficiencia a costa de una posible pérdida de precisión de la inferencia.

Batch size

- ▶ Define la cantidad de muestras que se procesan juntas en una sola iteración.
- ▶ Un batch size mayor puede incrementar la cantidad de inferencias realizadas por unidad de tiempo, pero también requiere más memoria.
- ▶ Se puede configurar un **batch size estático**:
 - La optimización del modelo se realizará exclusivamente para ese tamaño fijo.
 - El inference engine resultante solo podrá operar con dicho tamaño al ejecutar inferencias.
- ▶ Alternativamente, se puede optar por un **batch size dinámico**¹¹:
 - Permite al inference engine realizar inferencias con una cantidad variable de muestras de entrada dentro de un rango definido en la configuración¹².

¹¹Batch size de configuración: Indica si el inference engine fue configurado con un batch size estático o dinámico.

¹²La cantidad de muestras de entrada que el modelo procesa durante una inferencia.

Network definition

- ▶ La **building phase** comienza con la generación de una **network definition**.
- ▶ Es una representación interna exclusiva de TensorRT.
- ▶ Incluye información detallada sobre:
 - Tipos de capas (convolucionales, *fully-connected*, activación, etc.).
 - Parámetros de cada capa.
 - Definición de entradas y salidas de la red con sus dimensiones.
- ▶ Los tensores no marcados como entradas o salidas se consideran valores transitorios que el builder puede optimizar.

Network definition

- ▶ Se puede obtener automáticamente mediante el **parser integrado en TensorRT¹³**.
- ▶ El parser es compatible con el formato de grafo **ONNX**.
- ▶ Alternativamente, TensorRT permite generar una network definition de forma manual utilizando funciones como `layer` y `tensor`.
- ▶ **El proceso manual está escasamente documentado y puede ser molesto.**
- ▶ El método más común es a través del uso de **ONNX**.
- ▶ Luego de obtener la network definition, es necesario describir la **builder configuration**.

¹³ `parser` es un módulo que lee el grafo del modelo (por ejemplo, **ONNX**) y lo convierte automáticamente en una "network definition"

Memory

- ▶ Define el límite de memoria que cada capa del modelo puede utilizar durante el proceso de optimización.
- ▶ Se crean múltiples instancias del modelo, lo que incrementa los requerimientos de memoria.
- ▶ Se establece como la memoria máxima disponible en el dispositivo por defecto.
- ▶ Este parámetro aplica únicamente durante el diseño y generación del *engine*, no durante la inferencia.

Builder optimization level

- ▶ Define el nivel de optimización que el builder empleará para buscar y aplicar mejoras al modelo¹⁴.
- ▶ El comportamiento del builder según el nivel seleccionado:
 - **Nivel 0:** Habilita la compilación más rápida al deshabilitar la generación dinámica de kernels y seleccionar la primera táctica que se ejecute con éxito. No utiliza caché de tiempos.
 - **Nivel 1:** Ordena las tácticas disponibles mediante heurísticas y prueba únicamente las más prometedoras para seleccionar la óptima¹⁵.
 - **Nivel 2:** Ordena las tácticas disponibles por heurísticas, pero prueba únicamente las tácticas más rápidas para determinar la mejor.
 - **Nivel 3:** Aplica heurísticas para decidir si se utiliza un kernel precompilado estático o si es necesario compilar uno dinámicamente¹⁶.
 - **Nivel 4:** Siempre compila un kernel dinámico.
 - **Nivel 5:** Compila un kernel dinámico y lo compara con kernels estáticos.
- ▶ Un nivel más alto permite al *builder* probar una mayor variedad de combinaciones y tipos de *kernels*, a cambio de un mayor tiempo de optimización.

¹⁴El valor del parámetro es un número entero en el intervalo de 0 a 5.

¹⁵Si se genera un kernel dinámico, su nivel de optimización es bajo.

¹⁶kernel dinámico es un kernel que no está precompilado en una biblioteca estática, sino que se genera o ajusta en tiempo de optimización (*build time*) para adaptarse mejor a las características del modelo, el tamaño del *batch*, o el *hardware* específico.

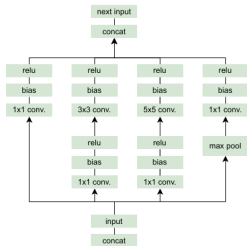
Iterations

- ▶ **Iterations:** Define la cantidad de veces que se ejecuta cada táctica o configuración de kernel durante la etapa de *automatic kernel tuning*.
- ▶ Por defecto, el *builder* evalúa el tiempo de ejecución de cada táctica una sola vez.
- ▶ Al incrementar el valor de *iterations*, se calcula un tiempo promedio, lo cual permite seleccionar la táctica más estable y rápida en promedio.
- ▶ Un mayor número de iteraciones reduce el impacto del ruido del sistema (por ejemplo, cambios en la frecuencia de la GPU o procesos en segundo plano).
- ▶ Ayuda a mejorar la convergencia del *inference engine*, disminuyendo el efecto del comportamiento no determinístico del sistema.

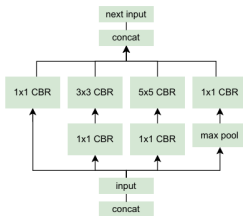
► El builder realiza optimizaciones como:

- Eliminar cálculos redundantes.
- Combinar y reordenar operaciones.
- Opcionalmente, reducir la precisión de los parámetros (fp32 a fp16 o cuantización de fp32 a int8).
- Evaluar el tiempo de ejecución de múltiples implementaciones para cada capa.

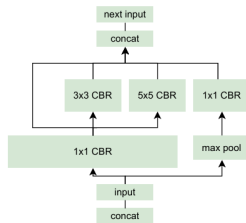
Modelo optimization



(a) Red neuronal.



(b) Fusión vertical de capas.



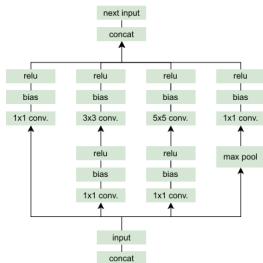
(c) Fusión horizontal de capas.

- ▶ **Dead layer removal**¹⁷
- ▶ **Vertical fusion:** Fusiona capas secuenciales con entradas similares (e.g., convolución, *bias*, ReLU) en una sola operación de kernel para reducir el número de lanzamientos.
- ▶ **Horizontal fusion:** Combina operaciones con parámetros similares sobre el mismo tensor de entrada en una única capa, reduciendo lanzamientos de kernel.
- ▶ **Quantization**¹⁸
- ▶ **Automatic kernel tuning:** Selecciona los kernels más eficientes durante la compilación, optimizando la ejecución.

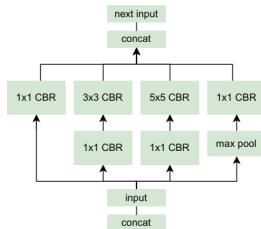
17 Elimina capas cuyas salidas no son utilizadas (**pesos con valores cercanos a cero**), reduciendo el modelo sin afectar significativamente la salida.

18 Reduce la precisión del modelo (e.g., de fp32 a int8) para disminuir el uso de recursos, manteniendo precisión mediante calibración automática.

Vertical Fusion



(a) Red neuronal.



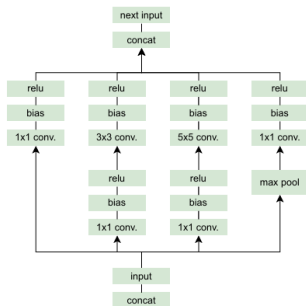
(b) Fusión vertical de capas.

- **Análisis de operaciones secuenciales:** En muchas arquitecturas de ANNs, las capas se organizan de manera secuencial. Una secuencia típica podría ser: **convolución** → **sesgo (bias)** → **activación ReLU**. En lugar de ejecutar cada operación como un kernel separado, TensorRT identifica estos patrones y los combina en una única operación¹⁹.
- **Reducción del *overhead* de ejecución:** Cada lanzamiento de kernel desde el host hacia el dispositivo (GPU) incurre en un costo adicional de configuración, sincronización y comunicación²⁰. Al fusionar operaciones en un único kernel, se reduce considerablemente este *overhead*, lo que se traduce en una latencia menor y un mayor rendimiento.

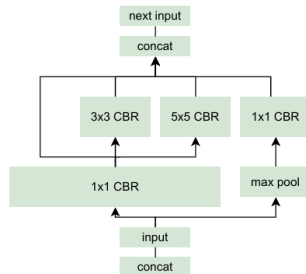
¹⁹Esto se logra siempre que las dimensiones y características de entrada/salida sean compatibles, permitiendo que un solo kernel realice múltiples operaciones de manera más eficiente.

²⁰Aunque este costo pueda parecer insignificante, se acumula en redes profundas con muchas capas.

Horizontal Fusion



(a) Red neuronal.

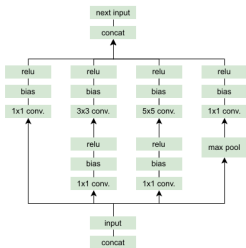


(b) Fusión horizontal de capas.

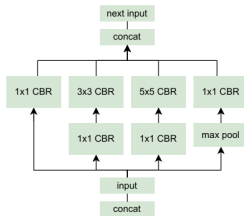
- **Combinación de capas paralelas con entradas comunes:** La fusión horizontal se aplica cuando múltiples capas reciben el mismo tensor de entrada y realizan operaciones equivalentes (por ejemplo, convoluciones con filtros 1×1 pero con diferentes pesos).
- En lugar de lanzar múltiples kernels, uno por cada capa, TensorRT agrupa estas operaciones en un solo kernel. Esto reduce el número de lanzamientos y mejora la eficiencia de cómputo.
- Tras la operación unificada, los resultados deben **separarse** para mantener la estructura original del grafo, de modo que cada rama posterior reciba su correspondiente salida.

Este proceso se ilustra en la Figura, donde tres capas CBR paralelas son fusionadas en una única operación.

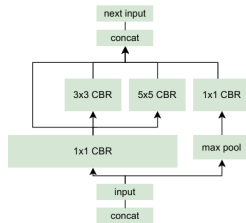
Automatic kernel tuning



(a) Red neuronal.



(b) Fusión vertical de capas.



(c) Fusión horizontal de capas.

- **Exploración y selección de kernels óptimos:** Durante esta etapa, el *builder* de TensorRT selecciona los kernels más adecuados para ejecutar cada operación del modelo, basándose en el nivel de optimización definido mediante el parámetro *builder optimization level*.
- **TensorRT posee una biblioteca de kernels especializados para diferentes tipos de capas y configuraciones, y el proceso de *tuning* consiste en evaluar iterativamente cuál kernel ofrece el mejor rendimiento en el hardware objetivo²¹.**
- **Naturaleza no determinística:** El proceso no es completamente determinista: optimizar el mismo modelo varias veces podría generar *engines* de inferencia distintos²².

²¹Esta evaluación toma en cuenta factores como el tamaño del *batch*, las dimensiones del filtro y la arquitectura de la GPU.

²²Dado que la medición del tiempo de ejecución se realiza en condiciones reales, el *tuning* puede verse afectado por factores externos como procesos en segundo plano o variaciones en la frecuencia de la GPU.

- ▶ **Deserialización del plan optimizado:** Se toma el plan generado durante la fase de optimización y se **deserializa para evitar optimizaciones en tiempo de ejecución**²³, permitiendo así una ejecución más eficiente.
- ▶ **Generación del inference engine:** A partir del plan deserializado, se construye el ***inference engine***, encargado de ejecutar el modelo de forma optimizada en el hardware objetivo.
- ▶ **Creación del contexto de ejecución:** Contiene toda la información necesaria para llevar a cabo inferencias, incluyendo referencias a la memoria del dispositivo, configuraciones de entrada y salida.
- ▶ **Ejecución de inferencias:** Una vez definido el contexto de ejecución, se especifican las entradas y se obtienen las salidas correspondientes. En plataformas CUDA, este contexto representa el entorno donde se ejecutan los kernels y se gestiona la memoria del dispositivo.

²³Se carga el plan previamente optimizado y guardado para evitar tener que hacer toda la optimización otra vez, lo cual ahorra tiempo y recursos. Al deserializarlo, se recupera el engine listo para hacer inferencias sin necesidad de volver a compilar o afinar el modelo.

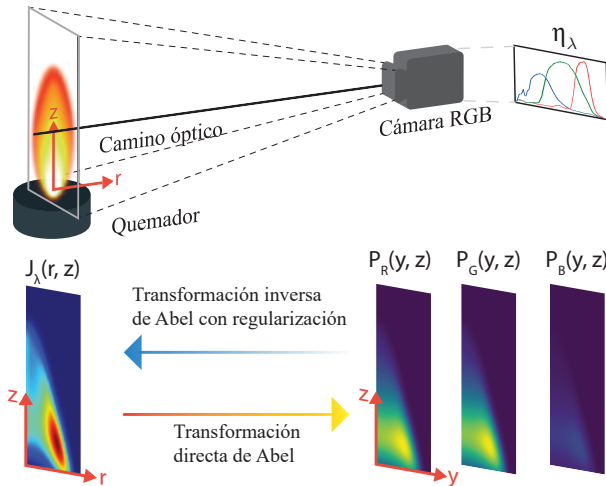
- ▶ Las plataformas Jetson permiten configurar el **modo de consumo energético**, el cual:
 - Controla el número de núcleos activos.
 - Establece un límite de consumo energético permitido.
- ▶ Estas configuraciones son independientes de las usadas por TensorRT.
- ▶ Se usaron los modos activos al iniciar el dispositivo (según configuración de fábrica), detallados en la siguiente tabla.
- ▶ Nota: Estos modos no necesariamente coinciden con los predeterminados de la documentación oficial²⁴.

Plataforma	Modo de Consumo	Power budget [W]	Online CPU
Jetson Xavier NX	Modo PM5	10	4
Jetson Xavier AGX	Modo PM7	15	4
Jetson Orin Nano	Modo PM0	15	6
Jetson Orin AGX	Modo PM2	30	8

²⁴<https://docs.nvidia.com/jetson/archives/r35.1/DeveloperGuide/text/SD/PlatformPowerAndPerformance/JetsonOrinNxSeriesAndJetsonAgxOrinSeries.html#supported-modes-and-power-efficiency>

Plano de simetría 2D de la
llama 3D CLAD

Proyección 2D de la llama 3D
captada por la cámara



El flujo de trabajo utilizado para obtener los *inference engines* derivados de los modelos base U-Net y Attention U-Net sigue el esquema PyTorch-ONNX-TensorRT.

Plataformas de hardware seleccionadas: una plataforma embebida Jetson Orin AGX y un computador de escritorio equipado con una tarjeta gráfica RTX 3060.

Códigos: <https://github.com/Juanx65/UNetTRT>

Tabla 1: Propiedades de los modelos U-Net y Attention (**Att.**) U-Net en sus distintas versiones: modelo base, TRT fp32, TRT fp16 y TRT int8.

Hardware	Modelo	Propiedades		
		Tamaño [MB]	# Capas	# Parámetros
RTX 3060	U-Net Base	7.25	91	1884631
	U-Net TRT fp32	10.75	23	1880040
	U-Net TRT fp16	4.45	25	1880040
	U-Net TRT int8	2.84	31	1880040
	Att. U-Net Base	33.75	114	8828070
	Att. U-Net TRT fp32	35.13	70	8823221
	Att. U-Net TRT fp16	18.57	46	8823221
	Att. U-Net TRT int8	11.06	48	8823221
Jetson Orin AGX	U-Net Base	7.25	91	1884631
	U-Net TRT fp32	7.88	24	1880040
	U-Net TRT fp16	4.66	25	1880040
	U-Net TRT int8	2.64	30	1880040
	Att. U-Net Base	33.75	114	8828070
	Att. U-Net TRT fp32	35.33	61	8823221
	Att. U-Net TRT fp16	18.39	46	8823221
	Att. U-Net TRT int8	9.75	30	8823221

Tabla 2: Resultados de **regression accuracy** en Kelvin (K) bajo diferentes condiciones de llama.

	Modelo	Cond. de llama	Modelo base	TRT fp32	TRT fp16	TRT int8
RTX 3060	U-Net	Yale-32	15.3	19.4	19.5	21.3
		Yale-40	40.8	39.8	39.6	36.7
		Yale-60	31.7	35.4	35.4	35.2
	Att. U-Net	Yale-32	31.0	31.0	31.0	29.9
		Yale-40	38.4	38.4	38.4	56.6
		Yale-60	28.4	28.4	28.4	41.6
J. Orin AGX	U-Net	Yale-32	15.3	19.4	19.5	19.6
		Yale-40	40.8	39.8	39.6	37.9
		Yale-60	31.7	35.4	35.4	33.9
	Att. U-Net	Yale-32	31.0	31.0	31.0	29.4
		Yale-40	38.4	38.4	38.4	49.5
		Yale-60	28.4	28.4	28.4	43.2

Soot Pyrometry

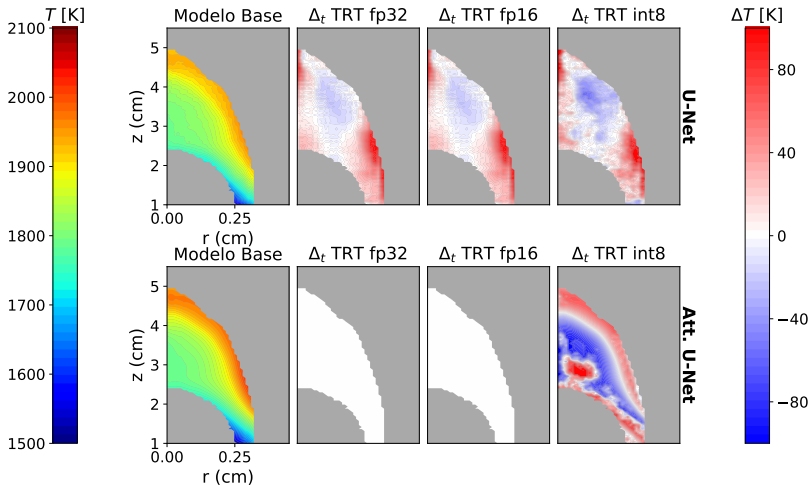


Figura 1: Comparación de la predicción de T_s entre el modelo base y los modelos optimizados bajo la condición de llama Yale-60. Se incluye el mapa de error ΔT respecto al modelo base.

► Classification closeness

- Evalúa el impacto de las optimizaciones de TensorRT sobre las salidas numéricas de las redes neuronales.
- Se utiliza `torch.isclose` para comparar las salidas:

$$|a - b| \leq \text{atol} + \text{rtol} \cdot |b|$$

- Se fija `rtol = 0`. Los valores de `atol` se definen como porcentajes del valor máximo dentro del percentil 90 de las salidas del modelo base.

► Uso de memoria durante la inferencia

- Evalúa el consumo de RAM durante todo el proceso de inferencia: carga, preprocesamiento, inferencia y postprocesamiento.
- Se emplea `tegrastat` con muestreo de 1 ms para registrar el uso de memoria.
- Se calcula el uso promedio y máximo de memoria para distintas configuraciones y valores de batch size.

Para las configuraciones fp32 y fp16, el modelo Attention U-Net alcanza un closeness del 100 % con una tolerancia del 0.5 %. En contraste, la red U-Net, bajo la misma condición de atol, solo alcanza un closeness del 2 %, lo que evidencia una mayor robustez de Attention U-Net frente a la optimización con cuantización fp32 y fp16.

Tabla 3: Resultados de closeness (%) según diferentes razones de tolerancia absoluta para los modelos U-Net y Attention U-Net.

			atol 0.005	atol 0.01	atol 0.1	atol 0.2	atol 0.5	atol 1
RTX 3060	U-Net	fp32	3.10 %	6.51 %	61.28 %	85.52 %	97.94 %	99.99 %
		fp16	3.10 %	6.43 %	61.36 %	85.51 %	97.94 %	99.99 %
		int8	3.46 %	6.92 %	62.53 %	85.27 %	97.08 %	99.98 %
	Att. U-Net	fp32	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		fp16	99.97 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		int8	4.53 %	8.94 %	73.67 %	93.21 %	99.87 %	100.00 %
J.O.AGX	U-Net	fp32	3.14 %	6.47 %	61.81 %	85.63 %	98.00 %	99.99 %
		fp16	3.16 %	6.47 %	61.82 %	85.62 %	98.00 %	99.99 %
		int8	3.11 %	5.65 %	51.87 %	82.60 %	98.62 %	99.97 %
	Att. U-Net	fp32	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		fp16	99.97 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		int8	2.55 %	4.99 %	37.79 %	59.58 %	91.32 %	99.96 %

Tabla 4: Resultados de latencia para un batch size de inferencia de 1 y de throughput para batch sizes de inferencia entre 2 y 32 en la plataforma Jetson Orin AGX.

	Modelo	Latencia [ms]		Throughput [inf/s]				
		ave.	max.	2	4	8	16	32
U-Net	Modelo Base	126.5	714.5	27.17	31.29	81.33	117.49	208.05
	TRT fp16	112.2	139.7	35.18	52.53	125.72	135.41	268.36
Att. U-Net	Modelo Base	134.2	753.8	15.20	30.97	58.53	92.60	122.94
	TRT fp16	115.4	312.8	19.74	42.10	65.11	123.34	220.30

With great power comes great responsibility.

AND, A SHORT DISTANCE AWAY...

MY FAULT--ALL MY FAULT! IF ONLY I HAD STOPPED HIM WHEN I **COULD** HAVE! BUT I **DIDN'T**--AND NOW --UNCLE BEN-- IS DEAD...

AND A LEAN, SILENT FIGURE SLOWLY FADES INTO THE GATHERING DARKNESS, AWARE AT LAST THAT IN THIS WORLD, WITH GREAT POWER THERE MUST ALSO COME -- GREAT RESPONSIBILITY!

AND SO A LEGEND IS BORN AND A NEW NAME IS ADDED TO THE ROSTER OF THOSE WHO MAKE THE WORLD OF FANTASY THE MOST EXCITING REALM OF ALL!

NEXT ISSUE OF **AMAZING FANTASY** --- FOR THE FURTHER AMERICA'S MOST **DIFFERENT** NEW TEEN-AGE IDOL -- **SPIDERMAN!**

the End



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Inference Optimizer

Knowledge distillation and NVIDIA TensorRT

Part II

Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile